AGENT SETUP GUIDE · V0.2.9

# Give any agent *a run button.*

Baton is one MCP server, `baton-run` on npm. This guide has the one command that sets up every coding agent at once, and for each agent the same entry by hand, exactly as `baton setup` writes it.

## Before you start

- **Node.js 24 or newer** on the machine that runs your apps: `node -v`. `nvm install 24` if not.
- **iOS:** Xcode and a simulator (on a Mac). **Android:** Android Studio and an emulator. Your agent can boot either with `boot_device`.
- **Web screenshots:** Chrome, Chromium, Edge or Brave installed, or `BATON_CHROME` set to a browser binary.

## Every agent, one command

```
$ npm install -g baton-run
$ baton setup
```

`baton setup` finds the agents installed on this machine and adds Baton to each. It edits only the `baton` entry, leaves everything else in the file alone, keeps a `.baton-backup` of each file it changes, and is safe to run again. Restart each app (or start a new chat) afterwards.

| COMMAND | DOES |
|---|---|
| `baton setup` | Every installed agent |
| `baton setup cursor codex` | Only the agents named. Ids: `claude` `codex` `cursor` `vscode` `antigravity` `gemini` `windsurf` |
| `baton setup --dry-run` | Show what would change; change nothing |
| `baton setup --remove` | Take Baton out again (add ids to limit it) |
| `npx -y --package=baton-run baton setup` | The same, without a global install |

It writes the absolute path of Node and this install, plus your shell's `PATH`, so apps started from the Dock still find Node, Flutter and Xcode. Run from `npx`, it writes `npx -y baton-run@0.2.9` instead.

AGENT BY AGENT

# Claude *and Codex.*

## Claude Code

the Baton plugin

One command `baton setup claude` runs `claude plugin install baton@baton` for you

BY HAND, INSIDE CLAUDE CODE

```
› /plugin marketplace add kanumuri9593/Baton
› /plugin install baton@baton
```

Claude Code gets the plugin, not a bare server: the `baton` MCP server, the `using-baton` skill that teaches the loop, `/baton:run`, `/baton:doctor`, the band above the prompt with a row of buttons per run (terminal and desktop app), and `/baton` to open it in a pane.

Server only, no plugin: `claude mcp add baton -- npx -y baton-run@0.2.9`

---

## Codex

~/.codex/config.toml

One command `baton setup codex`

RECOMMENDED: THE CODEX PLUGIN

```
$ codex plugin marketplace add kanumuri9593/Baton
$ codex plugin add baton@baton
```

Codex keeps MCP tools out of the prompt until it searches for them, so the plugin's skills (how to run, reload and check an app) are what make "run my app" reach Baton. The plugin brings the server with it.

BY HAND, SERVER ONLY

```
[mcp_servers.baton]
command = "npx"
args = ["-y", "baton-run@0.2.9"]
tool_timeout_sec = 900

[mcp_servers.baton.env]
PATH = "/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin"
```

`tool_timeout_sec = 900` gives long steps, like a first iOS build, 15 minutes. For `PATH`, paste the output of `echo $PATH` from your shell.

---

## Claude Desktop chat

claude_desktop_config.json · manual only

`baton setup` leaves it out on purpose: the running app rewrites `claude_desktop_config.json` and drops servers added behind its back. **Quit the app first**, add this, then reopen it.

```
{ "mcpServers": { "baton": {
  "command": "npx",
  "args": ["-y", "baton-run@0.2.9"]
} } }
```

AGENT BY AGENT

# Editors *and CLIs.*

Each entry below is the shape `baton setup` writes. `baton setup` fills in absolute paths; by hand, `npx` works. Set `PATH` to the output of `echo $PATH`.

## Cursor

`~/.cursor/mcp.json`

One command `baton setup cursor`

```
{ "mcpServers": { "baton": {
  "command": "npx",
  "args": ["-y", "baton-run@0.2.9"],
  "env": { "PATH": "…" },
  "timeout": 900000
} } }
```

Or `.cursor/mcp.json` in one project.

## VS Code (Copilot)

`User/mcp.json`

One command `baton setup vscode`

```
{ "servers": { "baton": {
  "type": "stdio",
  "command": "npx",
  "args": ["-y", "baton-run@0.2.9"],
  "env": { "PATH": "…" }
} } }
```

User folder: `~/Library/Application Support/Code/User` (macOS), `~/.config/Code/User` (Linux), `%APPDATA%\Code\User` (Windows). Or `.vscode/mcp.json` in a project. Claude Code in VS Code uses the plugin instead.

## Antigravity

`~/.gemini/antigravity/mcp_config.json`

One command `baton setup antigravity`

```
{ "mcpServers": { "baton": {
  "command": "npx",
  "args": ["-y", "baton-run@0.2.9"],
  "env": { "PATH": "…" },
  "timeout": 900000
} } }
```

## Gemini CLI

`~/.gemini/settings.json`

One command `baton setup gemini`

```
{ "mcpServers": { "baton": {
  "command": "npx",
  "args": ["-y", "baton-run@0.2.9"],
  "env": { "PATH": "…" },
  "timeout": 900000
} } }
```

## Windsurf

`~/.codeium/windsurf/mcp_config.json`

One command `baton setup windsurf`

```
{ "mcpServers": { "baton": {
  "command": "npx",
  "args": ["-y", "baton-run@0.2.9"],
  "env": { "PATH": "…" },
  "timeout": 900000
} } }
```

## Anything else

`any mcpServers config`

Any client that takes an `mcpServers` block takes the same entry. Raise its per-tool timeout if it has one: a first iOS build can take several minutes.

```
{ "mcpServers": { "baton": {
  "command": "npx",
  "args": ["-y", "baton-run@0.2.9"]
} } }
```

TEACH THE LOOP

# Paste this into your *agent's rules.*

Put it in `AGENTS.md`, `CLAUDE.md`, `GEMINI.md` or your rules file. (The Claude Code and Codex plugins already ship it as a skill.)

```
## Running the app
Use the Baton MCP tools, not raw `flutter run` / `npm run dev` in a shell.
1. `inspect_project` and `list_targets` with cwd set to the absolute project path.
2. `run_target` (or `run_workflow` for API + frontend), then `wait_for`.
   No simulator up? `boot_device` first.
3. After every edit: `check_change`. Fix anything it reports as NOT OK and
   call it again. For web apps pass `viewport: "phone"` to check the mobile
   layout too.
4. Look at the screenshot and say what you see. A capture is not proof the
   UI is right.
```

| TOOL | RETURNS |
|---|---|
| `inspect_project` | Runnable targets, blockers (a missing secrets file, a wrong Node) and guidance, before anything runs. |
| `boot_device` | Boots an iOS simulator or Android emulator by name and returns the id for `run_target`'s `deviceId`. No name: lists what can boot. |
| `run_target` · `run_workflow` | A session per app, with its URL or device, readiness and focused errors. |
| `wait_for` | Blocks until a session is running, has a URL, or prints a log line. Up to 15 minutes, enough for a first native build. |
| `check_change` | OK / NOT OK, the reload result, new error lines, failed requests and a screenshot. |

## No MCP? Use the CLI

```
$ baton list                              # what can run here
$ baton run dev
$ baton check dev --viewport phone --json  # same verdict as check_change
```

Every command exits non-zero on failure. Without a global install:
`npx -y --package=baton-run@0.2.9 baton <command>`.

## Pairs well with

Baton runs the app and reports what happened. It does not tap or type. When a check needs input, add **Playwright MCP** or **Chrome DevTools MCP** for web, pointed at the URL Baton returns, or a device-control MCP such as **mobile-mcp** for the simulator Baton booted.

TROUBLESHOOTING

# When it doesn't *just work.*

| YOU SEE | DO THIS |
|---|---|
| **No baton tools in the agent** | Restart the app or start a new chat after setup. Run `baton setup --dry-run` to see what is set up. In Claude Code, `/baton:doctor`; elsewhere `baton doctor`. |
| **Codex ignores "run my app"** | Install the Codex plugin (page 2). Codex only loads MCP tools once it searches for them; the plugin's skills point it at Baton. |
| **baton setup says "skipped"** | That file is not plain JSON (comments, for example). Add the entry from page 3 by hand. |
| **Baton vanished from Claude Desktop** | The app rewrote its config while running. Quit it, add the entry again, reopen. |
| **A Node error, or "Unsupported engine"** | Baton needs Node 24+. `nvm install 24 && nvm use 24`, run `baton setup` again so the entry points at the new Node, and restart the agent. |
| **The app from the Dock can't find flutter or xcrun** | Re-run `baton setup` from a shell where they work: it copies that shell's `PATH` into the entry. |
| **Tool timed out on a first iOS build** | The Codex, Cursor, Antigravity, Gemini CLI and Windsurf entries allow 15 minutes per tool. Elsewhere, raise the client's per-tool timeout. `wait_for` waits up to 15 minutes. |
| **"web screenshots need Chrome..."** | Install Chrome, Chromium, Edge or Brave, or set `BATON_CHROME` to the browser binary. |
| **run_target finds no device** | `boot_device` with a simulator name first, then `run_target` with its `deviceId`. |

## Uninstall

```
$ baton setup --remove        # every agent; add ids to limit it
$ npm uninstall -g baton-run
```

## Safety

The daemon listens on 127.0.0.1 only and authenticates with a token in `~/.baton/daemon.json` (mode 0600). Baton only runs targets from your own launch configs and package scripts. Don't expose the port, and keep secrets out of launch files you commit.



**github.com/kanumuri9593/Baton · docs/clients.md**

Free and open source (MIT) · Built by Yeswanth Varma Kanumuri · Issues and feedback welcome